

Homepage Map REST API

Status: contract (backend not implemented in this repo)

Version: v1

Base path: /api/v1/homepage-map

Format: JSON (application/json; charset=utf-8)

Source of truth in code: src/homepage-map/types.ts, src/homepage-map/ranking.ts, src/homepage-map/data/catalog.ts

This document is the API the frontend should call for destinations, cities, attractions, and products. Google Maps is a separate client-side SDK and is **not** part of this API.

Related: [HOMEPAGE_MAP_API.md](#) (architecture). This file is the endpoint-level contract.

Contents

1. [Overview](#)
 2. [Conventions](#)
 3. [Enums](#)
 4. [Shared schemas](#)
 5. [Ranking rules](#)
 6. [Catalog identifiers](#)
 7. [Error model](#)
 8. [Endpoints](#)
 - [GET /audiences](#)
 - [GET /destinations](#)
 - [GET /countries/{countryId}](#)
 - [GET /countries/{countryId}/cities](#)
 - [GET /cities/{cityId}](#)
 - [GET /cities/{cityId}/attractions](#)
 - [GET /cities/{cityId}/attractions/{attractionId}](#)
 - [GET /bootstrap](#)
 9. [When the frontend calls each endpoint](#)
 10. [Headers, CORS, caching](#)
 11. [Worked request flow](#)
-

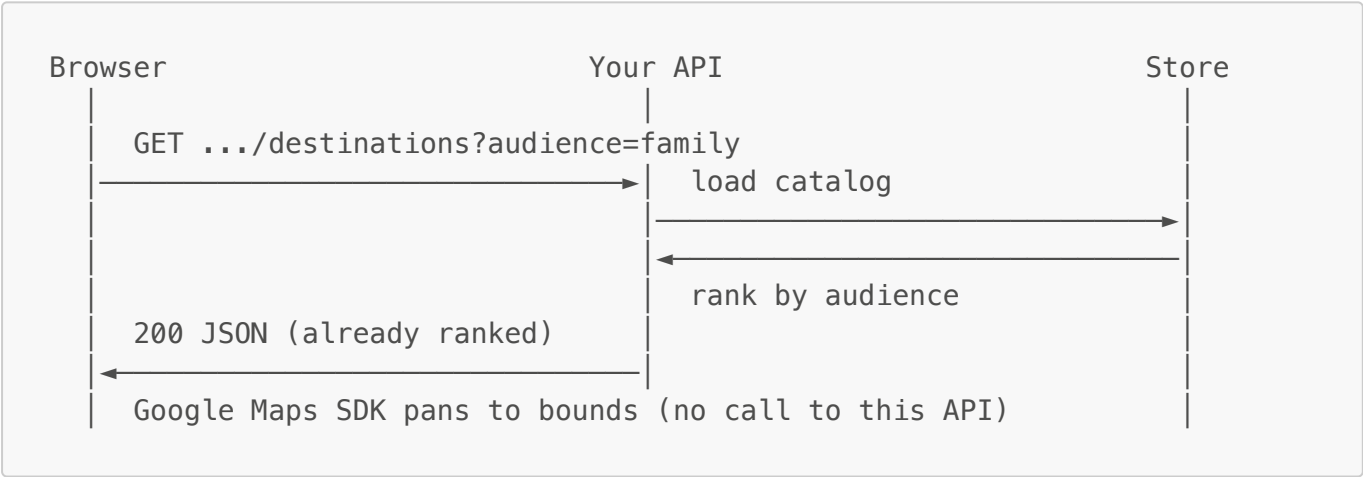
1. Overview

The homepage map has four zoom levels. Each level is one read endpoint (plus a bootstrap call on first load).

Map level	User sees	Endpoint
world	Ranked countries as pins + list	GET /destinations
country	Ranked cities in that country	GET /countries/{countryId}/cities
city	Ranked attractions in that city	GET /cities/{cityId}/attractions
poi	Products for one attraction	GET /cities/{cityId}/attractions/{attractionId}

The backend **ranks and copies** (tier, why, chips, trade-off). The frontend **places pins and flies the camera** using **lat / lng** from the JSON.

All endpoints are **GET**, idempotent, and safe to cache.



2. Conventions

Topic	Rule
Protocol	HTTPS in production. HTTP allowed on localhost.
Base URL (dev)	http://localhost:8080/api/v1/homepage-map
Base URL (prod)	https://{api-host}/api/v1/homepage-map
Frontend env	VITE_API_BASE_URL = origin only, e.g. http://localhost:8080
Method	GET only for v1
Auth	None for v1 (public catalog). Do not send cookies.
Query encoding	UTF-8. Always encodeURIComponent audience and ids.

Topic	Rule
Path ids	lowercase kebab-case or ISO country codes (pl , krakow , wawel-castle)
Numbers	JSON numbers, not strings. Prices are integers in EUR .
Coordinates	WGS84 decimal degrees. lat ∈ [-90, 90], lng ∈ [-180, 180].
Lists	Already sorted. Do not re-sort on the client.
Missing fit	If a destination/attraction has no fits entry for the requested audience, treat tier as 3 and generate fallback copy (see Ranking).
Currency	EUR. Prefix in UI is € . Do not send a currency symbol in JSON.
Locale	English copy in v1. No Accept-Language behaviour yet.
Pagination	None. Hard caps: destinations = all, cities ≤ 8, attractions ≤ 4, products ≤ 3.
Trailing slash	Do not use. /destinations not /destinations/ .
Unknown query keys	Ignore. Do not 400.
Default audience	family if audience is omitted.

Success envelope

Every **200** body includes:

```
{
  "apiVersion": "v1",
  "audience": "family"
}
```

List endpoints also include **count** (length of the primary array).

3. Enums

audience (query, required in practice)

Travel preference. Matches **AudienceId** in **types.ts**.

Value	UI label	Meaning
first	First visit	First-trip icons, skip-the-line, guided transit
family	Family	Short formats, shade, indoor options, under-12s
culture	Culture & history	Expert-led, restricted access, historian formats
active	Active	Early starts, boats, walking, outdoor days

Value	UI label	Meaning
<code>budget</code>	Budget-smart	Price per guided hour, included tastings, free grounds

Invalid value → `400` with `code: "INVALID_AUDIENCE"`.

`tier` (response integer)

Lower is better. Matches `TierIndex` and `TIERS` in `types.ts`.

<code>tier</code>	Label	When
<code>0</code>	Best match	Explicit fit for this audience at rank 0
<code>1</code>	Best alternative	Explicit fit at rank 1
<code>2</code>	Also fits	Explicit fit at rank 2
<code>3</code>	Worth considering	Explicit fit at rank 3, or no fit for this audience

Frontend maps `tier` to pin colour and `TierBadge`. Do not send the label; the client already has it.

`mapLevel` (informational on some responses)

`world` | `country` | `city` | `poi`

4. Shared schemas

Camera

Optional hint. Frontend can compute the same thing from pin coordinates (see `useHomepageMap.ts`). If present, the client may use it instead of deriving bounds.

```
{
  "center": { "lat": 50.0647, "lng": 19.945 },
  "zoom": 12,
  "bounds": {
    "north": 50.11,
    "south": 50.02,
    "east": 20.09,
    "west": 19.90
  }
}
```

Field	Type	Required	Notes
<code>center.lat</code>	number	yes	
<code>center.lng</code>	number	yes	
<code>zoom</code>	number	yes	World <code>2.4</code> , country <code>6.2</code> , city/poi <code>12</code>

Field	Type	Required	Notes
<code>bounds</code>	object	no	Used with <code>fitBounds</code> on the flat map

Pin

Minimum fields the map needs for a marker.

Field	Type	Required	Notes
<code>id</code>	string	yes	Stable id for React keys and next request
<code>name</code>	string	yes	Pin label
<code>lat</code>	number	yes	
<code>lng</code>	number	yes	
<code>tier</code>	0–3	yes	Pin colour
<code>count</code>	integer	no	Badge on the pill (picks or product count)
<code>selected</code>	boolean	no	Highlight (default city, or selected POI)
<code>rank</code>	integer	yes	0-based position in the ranked list

Product

Matches `Product` in `types.ts`. Shown on the POI panel.

Field	Type	Required	Example
<code>id</code>	string	yes	<code>wawel-castle-family-walk</code>
<code>title</code>	string	yes	<code>Wawel Castle & Cathedral, 90-minute family walk</code>
<code>rating</code>	number	yes	<code>4.9</code> (one decimal)
<code>reviews</code>	integer	yes	<code>1204</code>
<code>duration</code>	string	yes	<code>1h 30m, 2h 45m, 3h</code>
<code>price</code>	integer	yes	<code>32</code> (EUR, no cents in v1)
<code>why</code>	string	yes	Why this product fits the audience
<code>trade</code>	string	yes	Honest trade-off
<code>snip</code>	string	yes	Review-filter explanation
<code>quote</code>	string	yes	Traveller quote (no wrapping quotes in JSON)

Deduplicate products by `duration` + `price` before returning. Cap at **3**.

5. Ranking rules

The API must apply the same logic as `src/homepage-map/ranking.ts`. The client must **not** re-rank.

Destinations (world)

1. Read `fits[audience]`. If missing, `tier = 3`.
2. Sort by `tier` ascending, then `picks` descending.
3. Return **all** destinations (currently 6). Do not hide poor fits — the UI keeps them on the map, labelled honestly.

Fallback copy when `fits[audience]` is missing:

- `why`: Strongest for {bestAudience} rather than {requested} – {bestWhy}
- `tradeOff`: Inventory here is built around {bestAudience}, so expect fewer {requested} formats.
- `chips`: ["Best for {bestAudience}"]

`bestAudience` = the audience key on that destination with the lowest tier.

Cities (country)

1. Filter `city.countryId === countryId`.
2. The destination's `cityId` (default base) is always first.
3. Remaining cities sort by `picks` descending.
4. Cap at **8**.
5. City `tier` is **positional**, not from `fits`:

Position in ranked list	<code>tier</code>
0 (default base)	<code>0</code>
1	<code>1</code>
2	<code>2</code>
3+	<code>3</code>

Country `why` / `chips` / `tradeOff` are still taken from the **destination** fit for the requested audience (the city list reuses country copy).

Attractions (city)

1. Sort by attraction `tier` for this audience (missing fit → 3), then `reviews` descending.
2. Cap at **4**.
3. `factChip` = `Clear {shortestProductDuration} format` (parse `Xh Ym / Xh`).

Fallback `why` when no fit:

Prioritised for {bestAudience} rather than {requested} – {bestWhy}

Products (POI)

1. Unique by `duration|price`.

2. Keep original order after dedupe.
3. Cap at **3**.

Matchline (world header)

{n} best match(es) and {m} best alternative(s) for {audienceLabel}.
Everything else stays on the map, labelled honestly.

- **n** = destinations with **tier** === 0
- **m** = destinations with **tier** === 1
- Use singular **1 best match** / **1 best alternative** when the count is 1.

Audience badge count (preference bar)

Count of destinations where **fits[audience]** exists **and tier** <= 1.

This is **not** the same as "tier ≤ 1 after fallback". Destinations with no fit do not increment the badge.

Audience change → reset to world

If the user is not on **world** and the current country's **tier** for the **new** audience is > 2, the frontend resets to world. The destinations payload must include **tier** for every country so the client can do this without an extra call.

6. Catalog identifiers

Stable ids the API must accept. Do not use display names in URLs.

Countries (**countryId**)

id	name	default cityId
pl	Poland	krakow
it	Italy	rome
jp	Japan	kyoto
pt	Portugal	lisbon
gr	Greece	santorini
th	Thailand	bangkok

Cities (**cityId**)

Primary: **krakow**, **rome**, **kyoto**, **lisbon**, **santorini**, **bangkok**.

Also: **warsaw**, **gdansk**, **wroclaw**, **zakopane**, **florence**, **venice**, **naples**, **milan**, **tokyo**, **osaka**, **kanazawa**, **nara**, **porto**, **lagos**, **coimbra**, **athens**, **crete**, **naxos**, **chiangmai**, **phuket**, **ayutthaya**.

Attractions (**attractionId**)

Kebab-case slug of the display name. Examples:

cityId	attractionId	name
krakow	wawel-castle	Wawel Castle
krakow	kazimierz	Kazimierz
krakow	wieliczka-salt-mine	Wieliczka Salt Mine

Unknown id → 404 with **code**: "NOT_FOUND".

7. Error model

Every non-2xx body:

```
{
  "apiVersion": "v1",
  "error": {
    "code": "INVALID_AUDIENCE",
    "message": "audience must be one of: first, family, culture, active, budget",
    "param": "audience"
  }
}
```

HTTP	code	When
400	INVALID_AUDIENCE	audience not in the enum
400	INVALID_ID	Path id has illegal characters
404	NOT_FOUND	Unknown country, city, or attraction
405	METHOD_NOT_ALLOWED	Anything other than GET
429	RATE_LIMITED	Optional; include Retry-After
500	INTERNAL	Unexpected server error
503	UNAVAILABLE	Catalog store down

Do not leak stack traces. **param** is omitted unless the error is a query/path field.

8. Endpoints

8.1 List audiences

Returns the preference bar: labels plus match counts for **every** audience in one call.


```
GET /api/v1/homepage-map/audiences
```

Query: none.

When: page load. Counts do not depend on the selected audience.

200 OK

```
{
  "apiVersion": "v1",
  "audiences": [
    { "id": "first", "label": "First visit", "matchCount": 2 },
    { "id": "family", "label": "Family", "matchCount": 2 },
    { "id": "culture", "label": "Culture & history", "matchCount": 3 },
    { "id": "active", "label": "Active", "matchCount": 1 },
    { "id": "budget", "label": "Budget-smart", "matchCount": 3 }
  ]
}
```

Field	Type	Notes
audiences	array	Fixed order as above
audiences[].id	enum	
audiences[].label	string	UI chip text
audiences[].matchCount	integer	Destinations with explicit fit and tier ≤ 1

matchCount example for **family**: Poland (tier 0) + Italy (tier 1) → 2. Thailand family is tier 2, so it does **not** count.

8.2 List destinations (world)

Ranked countries for one audience. Drives world pins, the destinations list, and the matchline.

```
GET /api/v1/homepage-map/destinations?audience=family
```

Query	Type	Required	Default	Description
audience	enum	no	family	Preference used for ranking and copy

When: first load; user changes audience while on world; user zooms out to world; audience change resets to world.

200 OK

```
{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "world",
  "matchline": "1 best match and 1 best alternative for family. Everything
else stays on the map, labelled honestly.",
  "count": 6,
  "camera": {
    "center": { "lat": 30, "lng": 40 },
    "zoom": 2.4,
    "bounds": {
      "north": 60.2,
      "south": 6.5,
      "east": 109,
      "west": -16
    }
  },
  "destinations": [
    {
      "id": "pl",
      "name": "Poland",
      "lat": 52,
      "lng": 19.4,
      "cityId": "krakow",
      "cityName": "Kraków",
      "picks": 9,
      "opts": 180,
      "tier": 0,
      "rank": 0,
      "chips": [
        "Strong family-compatible inventory",
        "Shorter experience formats"
      ],
      "why": "Compact old towns you can walk in an hour, and an underground
afternoon that works in any weather.",
      "tradeOff": "Fewer English-language departures than western Europe –
the 10:00 slots go first."
    },
    {
      "id": "it",
      "name": "Italy",
      "lat": 42.8,
      "lng": 12.6,
      "cityId": "rome",
      "cityName": "Rome",
      "picks": 31,
      "opts": 812,
      "tier": 1,
      "rank": 1,
      "chips": ["Clear 3-hour format", "Air-conditioned interiors"],
      "why": "Short, early and indoors by noon – the only version of Rome
that works with children.",

```

```
    "tradeOff": "Fewer child discounts than northern Europe."
  }
]
```

Destination object

Field	Type	Required	UI use
<code>id</code>	string	yes	Next call: <code>/countries/{id}/cities</code>
<code>name</code>	string	yes	Pin label, list title
<code>lat, lng</code>	number	yes	Pin + world camera
<code>cityId</code>	string	yes	Default city when opening the country
<code>cityName</code>	string	yes	Subtitle: <code>{name} · {cityName}</code>
<code>picks</code>	integer	yes	<code>{picks}</code> picks from <code>{opts}</code>
<code>opts</code>	integer	yes	Total options in inventory
<code>tier</code>	0–3	yes	Badge + pin colour
<code>rank</code>	integer	yes	Pin z-index
<code>chips</code>	string[]	yes	Show first chip only in the list
<code>why</code>	string	yes	Body copy
<code>tradeOff</code>	string	yes	Not shown on world list; needed after drill-in

Pin `count` on the map = `picks`.

Errors

- `400 INVALID_AUDIENCE`

8.3 Get country

Country metadata plus suggested camera. Optional if `GET .../cities` already embeds `country`. Included so the client can refresh copy after an audience change without refetching cities.

```
GET /api/v1/homepage-map/countries/{countryId}?audience=family
```

Path	Type	Example
<code>countryId</code>	string	<code>pl</code>

Query	Default
-------	---------

audience	family
----------	--------

200 OK

```
{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "country",
  "country": {
    "id": "pl",
    "name": "Poland",
    "lat": 52,
    "lng": 19.4,
    "cityId": "krakow",
    "cityName": "Kraków",
    "picks": 9,
    "opts": 180,
    "tier": 0,
    "chips": [
      "Strong family-compatible inventory",
      "Shorter experience formats"
    ],
    "why": "Compact old towns you can walk in an hour, and an underground afternoon that works in any weather.",
    "tradeOff": "Fewer English-language departures than western Europe – the 10:00 slots go first."
  }
}
```

Errors

- 400 INVALID_AUDIENCE
- 404 NOT_FOUND — unknown `countryId`

8.4 List cities in a country

Ranked cities (max 8). Drives country pins and the city list.

```
GET /api/v1/homepage-map/countries/{countryId}/cities?audience=family
```

When: user clicks a country pin or row (`goCountry`).

200 OK

```

{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "country",
  "count": 5,
  "flash": "Poland · 5 cities prioritised",
  "camera": {
    "center": { "lat": 50.0647, "lng": 19.945 },
    "zoom": 6.2,
    "bounds": {
      "north": 55.15,
      "south": 48.50,
      "east": 21.81,
      "west": 18.15
    }
  },
  "country": {
    "id": "pl",
    "name": "Poland",
    "lat": 52,
    "lng": 19.4,
    "cityId": "krakow",
    "cityName": "Kraków",
    "picks": 9,
    "opts": 180,
    "tier": 0,
    "chips": [
      "Strong family-compatible inventory",
      "Shorter experience formats"
    ],
    "why": "Compact old towns you can walk in an hour, and an underground
afternoon that works in any weather.",
    "tradeOff": "Fewer English-language departures than western Europe –
the 10:00 slots go first."
  },
  "cities": [
    {
      "id": "krakow",
      "countryId": "pl",
      "name": "Kraków",
      "lat": 50.0647,
      "lng": 19.945,
      "picks": 9,
      "tier": 0,
      "rank": 0,
      "selected": true,
      "isDefaultBase": true
    },
    {
      "id": "warsaw",
      "countryId": "pl",
      "name": "Warsaw",

```

```

    "lat": 52.2297,
    "lng": 21.0122,
    "picks": 6,
    "tier": 1,
    "rank": 1,
    "selected": false,
    "isDefaultBase": false
  }
]
}

```

City object

Field	Type	Required	UI use
<code>id</code>	string	yes	Next call: <code>/cities/{id}/attractions</code>
<code>countryId</code>	string	yes	Breadcrumb / zoom out
<code>name</code>	string	yes	Pin + list
<code>lat, lng</code>	number	yes	Pin + country camera
<code>picks</code>	integer	yes	<code>{picks} picks</code> ; pin count
<code>tier</code>	0–3	yes	Positional (see ranking)
<code>rank</code>	integer	yes	
<code>selected</code>	boolean	yes	<code>true</code> for rank 0
<code>isDefaultBase</code>	boolean	yes	Shows "· default base"

List header: `{country.name} · {count} cities`.

List body `why` / first chip: from `country`, not from the city.

Trade-off box: `country.tradeOff`.

Toast (`flash`): `{country.name} · {count} cities prioritised`.

Camera: center on the default-base city; bounds pad country + all returned cities (~0.8°).

Errors

- `400 INVALID_AUDIENCE`
- `404 NOT_FOUND` — unknown `countryId`

8.5 Get city

```
GET /api/v1/homepage-map/cities/{cityId}?audience=family
```

Optional companion to the attractions list. Same `city` object as in §8.6.

Errors

- **404 NOT_FOUND** — unknown `cityId`

8.6 List attractions in a city

Ranked attractions (max 4). Drives city pins and the attractions list.

```
GET /api/v1/homepage-map/cities/{cityId}/attractions?audience=family
```

When: user clicks a city pin or row (`goCity`).

200 OK

```
{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "city",
  "count": 4,
  "flash": "Kraków · 4 attractions prioritised",
  "camera": {
    "center": { "lat": 50.0647, "lng": 19.945 },
    "zoom": 12,
    "bounds": {
      "north": 50.11,
      "south": 49.94,
      "east": 20.09,
      "west": 19.90
    }
  },
  "country": {
    "id": "pl",
    "name": "Poland",
    "tier": 0,
    "tradeOff": "Fewer English-language departures than western Europe – the 10:00 slots go first."
  },
  "city": {
    "id": "krakow",
    "countryId": "pl",
    "name": "Kraków",
    "lat": 50.0647,
    "lng": 19.945,
    "picks": 9
  },
  "attractions": [
    {
      "id": "wawel-castle",
      "name": "Wawel Castle",
```

```
    "category": "Castles & royal sites",
    "lat": 50.054,
    "lng": 19.9354,
    "from": 14,
    "rating": 4.8,
    "reviews": 3420,
    "productCount": 2,
    "tier": 0,
    "rank": 0,
    "selected": false,
    "factChip": "Clear 1h 30m format",
    "why": "Flat courtyards, 90-minute formats and a dragon story that
lands with children."
  },
  {
    "id": "wieliczka-salt-mine",
    "name": "Wieliczka Salt Mine",
    "category": "Underground & day trips",
    "lat": 49.983,
    "lng": 20.054,
    "from": 26,
    "rating": 4.7,
    "reviews": 5610,
    "productCount": 2,
    "tier": 0,
    "rank": 1,
    "selected": false,
    "factChip": "Clear 2h 30m format",
    "why": "An underground afternoon that works in any weather, with a
lift back up."
  }
]
```

Attraction list object

Field	Type	Required	UI use
id	string	yes	Next call: <code>.../attractions/{id}</code>
name	string	yes	Pin label, title
category	string	yes	Subtitle: <code>{name} · {category}</code>
lat, lng	number	yes	Pin + city camera
from	integer	yes	<code>from €{from}</code> (lowest product price)
rating	number	yes	<code>★ {rating}</code>
reviews	integer	yes	<code>({reviews})</code> — format with <code>en-US</code> grouping
productCount	integer	yes	Pin <code>count</code>

Field	Type	Required	UI use
tier	0–3	yes	Badge + pin
rank	integer	yes	
selected	boolean	yes	Always false at city level
factChip	string	yes	Chip next to the badge
why	string	yes	Body copy

Do **not** embed full `products` on this list. Fetch them on POI open.

List header: `{city.name} · {count} attractions`.

Trade-off box: `country.tradeOff` (country-level, not per attraction).

Toast: `{city.name} · {count} attractions prioritised`.

Camera: center on the city; bounds pad city + returned attractions (~0.04°).

Errors

- 400 INVALID_AUDIENCE
- 404 NOT_FOUND — unknown `cityId`

8.7 Get attraction (POI)

Products for one attraction (max 3 after dedupe).

```
GET /api/v1/homepage-map/cities/{cityId}/attractions/{attractionId}?
audience=family
```

When: user clicks an attraction pin or row (`goPoi`).

200 OK

```
{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "poi",
  "flash": "Wawel Castle · 2 products from €14",
  "camera": {
    "center": { "lat": 50.0647, "lng": 19.945 },
    "zoom": 12
  },
  "city": {
    "id": "krakow",
    "countryId": "pl",
    "name": "Kraków",
```

```
    "lat": 50.0647,
    "lng": 19.945
  },
  "attraction": {
    "id": "wawel-castle",
    "name": "Wawel Castle",
    "category": "Castles & royal sites",
    "lat": 50.054,
    "lng": 19.9354,
    "from": 14,
    "rating": 4.8,
    "reviews": 3420,
    "tier": 0,
    "why": "Flat courtyards, 90-minute formats and a dragon story that
lands with children."
  },
  "count": 2,
  "products": [
    {
      "id": "wawel-castle-family-walk",
      "title": "Wawel Castle & Cathedral, 90-minute family walk",
      "rating": 4.9,
      "reviews": 1204,
      "duration": "1h 30m",
      "price": 32,
      "why": "Ninety minutes, flat ground and a guide who tells the dragon
legend properly before the royal tombs.",
      "trade": "Skips the Crown Treasury – the room where most family
groups lose attention.",
      "snip": "Reviews shown are from travellers with children under 12.
61% of them mention the 90-minute length.",
      "quote": "Short enough that our seven-year-old was still listening at
the end."
    },
    {
      "id": "wawel-castle-historian",
      "title": "Royal apartments & Crown Treasury, historian-led",
      "rating": 4.8,
      "reviews": 876,
      "duration": "2h 45m",
      "price": 58,
      "why": "Full interior circuit with an art historian, including the
treasury and the Sigismund bell.",
      "trade": "Longer duration and 140 stairs – we would not book this one
with young children.",
      "snip": "Reviews shown are from travellers who booked expert-led
formats. Depth of commentary is the most-cited theme.",
      "quote": "The guide answered questions I did not know I had."
    }
  ]
}
```

POI panel mapping

UI	Field
Heading	<code>{attraction.name} · {count} product(s)</code>
Badge	<code>attraction.tier</code>
Product title	<code>product.title</code>
Meta line	<code>★ {rating} ({reviews}) · {duration} · €{price}</code>
Why this fits	<code>product.why</code>
Trade-off	<code>product.trade</code>
Quote	<code>product.quote</code>
Review context	<code>Showing {audience label} travellers because {attraction.why} {product.snip}</code>
CTA	<code>Check dates · from €{product.price}</code>
Toast	<code>{attraction.name} · {count} products from €{attraction.from}</code>

Camera stays at **city** zoom (same as `cityCamera` in the frontend). Do not zoom to the single POI in v1.

Pins on the map should still show the city's ranked attractions, with `selected: true` on this `attractionId`. The client can reuse the previous attractions response and set `selected`, or you may add `?includePins=true` later. v1: client reuses the city list.

Errors

- `400 INVALID_AUDIENCE`
- `404 NOT_FOUND` — unknown `cityId` or `attractionId`, or attraction not in that city

8.8 Bootstrap (first paint)

One round-trip for the preference bar + world view.

```
GET /api/v1/homepage-map/bootstrap?audience=family
```

When: `HomepageMap` mounts. After this, use the granular endpoints for drill-in.

200 OK

```
{
  "apiVersion": "v1",
  "audience": "family",
  "mapLevel": "world",
```

```
"audiences": [
  { "id": "first", "label": "First visit", "matchCount": 2 },
  { "id": "family", "label": "Family", "matchCount": 2 },
  { "id": "culture", "label": "Culture & history", "matchCount": 3 },
  { "id": "active", "label": "Active", "matchCount": 1 },
  { "id": "budget", "label": "Budget-smart", "matchCount": 3 }
],
"matchline": "1 best match and 1 best alternative for family. Everything
else stays on the map, labelled honestly.",
"count": 6,
"camera": {
  "center": { "lat": 30, "lng": 40 },
  "zoom": 2.4
},
"destinations": []
}
```

audiences is identical to §8.1. **destinations** / **matchline** / **camera** are identical to §8.2.

If bootstrap is not implemented, the client must call §8.1 and §8.2 in parallel.

9. When the frontend calls each endpoint

Assume **VITE_API_BASE_URL**=`http://localhost:8080`.

User action	Hook	Request
Open homepage	mount	GET /bootstrap?audience=family
Switch audience on world	setAud	GET /destinations?audience={id} (audiences counts can stay cached)
Switch audience on country	setAud	If new country tier > 2 → treat as world. Else GET /countries/{id}/cities?audience=
Switch audience on city	setAud	Same reset rule, else GET /cities/{id}/attractions?audience=
Switch audience on POI	setAud	Same reset rule, else refetch POI with new audience
Click country	goCountry	GET /countries/{countryId}/cities?audience=
Click city	goCity	GET /cities/{cityId}/attractions?audience=
Click attraction	goPoi	GET /cities/{cityId}/attractions/{attractionId}?audience=

User action	Hook	Request
Breadcrumb World	backWorld	Reuse cached destinations or refetch
Breadcrumb country	backCountry	Reuse cached cities or refetch
Breadcrumb city	backCity	Reuse cached attractions or refetch
Zoom in	zoomIn	Same as clicking the rank-0 item at the current level
Zoom out	zoomOut	Same as the matching breadcrumb

Example client

```
const BASE = `${import.meta.env.VITE_API_BASE_URL}/api/v1/homepage-map`

async function apiGet<T>(path: string): Promise<T> {
  const res = await fetch(`${BASE}${path}`, {
    headers: { Accept: 'application/json' },
  })
  if (!res.ok) {
    const body = await res.json().catch(() => null)
    throw new Error(body?.error?.message ?? `HTTP ${res.status}`)
  }
  return res.json() as Promise<T>
}

export const homepageMapApi = {
  bootstrap: (audience: string) =>
    apiGet(`/bootstrap?audience=${encodeURIComponent(audience)}`),
  destinations: (audience: string) =>
    apiGet(`/destinations?audience=${encodeURIComponent(audience)}`),
  cities: (countryId: string, audience: string) =>
    apiGet(
      `/countries/${encodeURIComponent(countryId)}/cities?`
      `audience=${encodeURIComponent(audience)}`,
    ),
  attractions: (cityId: string, audience: string) =>
    apiGet(
      `/cities/${encodeURIComponent(cityId)}/attractions?`
      `audience=${encodeURIComponent(audience)}`,
    ),
  attraction: (cityId: string, attractionId: string, audience: string) =>
    apiGet(
      `/cities/${encodeURIComponent(cityId)}/attractions/${encodeURIComponent(attractionId)}?audience=${encodeURIComponent(audience)}`,
    ),
}
```

Loading: keep the last pins on screen until the new payload arrives, then swap. On error, keep last good state and show a toast (do not blank the map).

10. Headers, CORS, caching

Request

```
GET /api/v1/homepage-map/destinations?audience=family HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: public, max-age=60, stale-while-revalidate=300
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET, OPTIONS
Access-Control-Allow-Headers: Accept
Vary: Origin
```

Production **Access-Control-Allow-Origin** must be the real site origin, not *****, if you later add credentials.

Preflight: respond to **OPTIONS** with **204**.

Caching

Catalog is public and changes slowly. **max-age=60** is enough for v1. Vary on the full URL (audience is in the query string).

11. Worked request flow

Family user opens the map, opens Poland, opens Kraków, opens Wawel Castle.

1. GET /bootstrap?audience=family
→ preference bar + 6 country pins, Poland tier 0
2. GET /countries/pl/cities?audience=family
→ Kraków selected, Warsaw / Gdańsk / Wrocław / Zakopane
→ camera zoom 6.2
3. GET /cities/krakow/attractions?audience=family
→ up to 4 attractions, Wawel Castle rank 0
→ camera zoom 12
4. GET /cities/krakow/attractions/wawel-castle?audience=family
→ 2 products, CTA from €32 / €58
→ camera stays at city

User then taps **Budget-smart**:

5. GET /destinations?audience=budget
(or cities/attractions if still drilled in and Poland budget tier is 0)
→ Poland still tier 0, stays in-country if you refetch cities
→ matchline and chips change

User then taps **Active** while still on Poland:

6. Destinations payload (or country object) has Poland tier 3 for active
→ frontend resets to world (tier > 2)
→ GET /destinations?audience=active

Endpoint index

Method	Path	Level
GET	/api/v1/homepage-map/bootstrap	world + audiences
GET	/api/v1/homepage-map/audiences	preference bar
GET	/api/v1/homepage-map/destinations	world
GET	/api/v1/homepage-map/countries/{countryId}	country meta
GET	/api/v1/homepage-map/countries/{countryId}/cities	country
GET	/api/v1/homepage-map/cities/{cityId}	city meta
GET	/api/v1/homepage-map/cities/{cityId}/attractions	city
GET	/api/v1/homepage-map/cities/{cityId}/attractions/{attractionId}	poi

Out of scope for v1: search, booking, availability, auth, webhooks, Google Places, photo URLs (UI uses empty photo slots).